Ray Tracing + ML Pipeline

Model Reference: Sionna 0.19 Calibration → Sionna 2 DEM

Scene	Nottingham, UK
Frequency	915.95 MHz (Ofcom 2018 drive-test)
TX position	Lat 52.9863 / Lon –1.2559, height 25 m AGL
Dataset	1,200 receivers | 120 calib receivers | 173 mat-calib receivers
Branch	claude/cool-cori-rrWbY
Date	June 2026

1. Pipeline Overview

The pipeline runs in two stages. **Stage 1 (Calibration)** runs in *sionna019_differentiable_rt_fixed.ipynb* using Sionna 0.19 differentiable RT. It calibrates physical and statistical models against drive-test receivers. **Stage 2 (Simulation)** runs in *sionna2_915mhz_dem_simulation.ipynb* using Sionna 2.0 with a Digital Elevation Model (DEM). It loads exported parameters from Stage 1 and produces final RSSI maps and RMSE results.

Cell	Model	Output file	Loaded in Sionna 2 DEM?
Cell 10b	Scalar offset (+5.80 dB)	scalar_offset_915mhz.json	■ Cell 4A — automatic
Cell 11b	Per-material εr, σ, S (NVLabs)	calibrated_materials_915mhz.json	■ Cell 4A — automatic
Cell 16	MaterialMLP (portable)	material_mlp_weights.npz	■■ Cell 4A — needs wiring
Cell 15	Residual MLP (Thrane)	residual_mlp_915mhz.npz	■■ Post-process — needs Sionna 2 API adapter
Cell 14	CNN+MLP (nDSM)	cnn_residual_915mhz.h5	■ Post-process — fully portable

2. Model Descriptions

2.1 Cell 10b — Global Scalar Offset

Property	Detail
Type	Single tf.Variable — global dB shift applied to all simulated RSSI
Input	rssi_sim (120 calib_receivers), rssi_meas (drive-test Ofcom 2018)
Method	SMAPE loss, 200 steps, Adam LR=0.10. Best-RMSE checkpoint saved.
Output	scalar_offset_915mhz.json → {"scaling_factor_db": +5.8024}
RMSE	7.90 dB (N=120 receivers, full calibration set)
In Sionna 2	Cell 4A reads JSON and adds offset to every simulated RSSI value
Portable?	■ No — must be re-run per scene and frequency

2.2 Cell 11b — Differentiable RT Material Calibration (NVLabs)

Property	Detail
Type	Per-material εr, σ, S calibration — 102 tf.Variables (14 active)
Input	173 receivers (0–1.2 km), drive-test RSSI. Ray depth = 8.
Method	MAE loss on RSSI. Tikhonov λ=0.001. 500 steps, cosine LR 0.05→0.001. PRUNE_ZERO_GRADS=False. Aligned with Hoydis et al. 2023 (NVLabs diff-rt-calibration).
εr bounds	max(1.0, εr_default×0.5) → min(20.0, εr_default×3.0) [NVLabs Table 1]
Output	cell11b_calibrated_materials.json → {"concrete": {"er":X,"sigma":Y,"s":Z}, ...}
Active mats	concrete, brick, glass, metal, asphalt — only materials visible to rays
RMSE	22.62 dB → 21.94 dB (+0.67 dB gain). Ceiling set by OSM geometry errors.
In Sionna 2	Cell 4A sets relative_permittivity / conductivity / scattering_coefficient on each RadioMaterial before trace_paths()
Portable?	■■ Limited — values tuned for Nottingham at 915 MHz

2.3 Cell 16 — MaterialMLP (Scene-Conditioned, Portable)

Property	Detail
Type	Neural network predicting εr, σ, S from 8 scene features — works for any city/frequency
Architecture	Input(n_mat one-hot 6×6) + Input(8 scene feats) → FC(64,ReLU)+BN → FC(32,ReLU)+BN → 3 output heads: eps_r [1–100], log_sigma [exp clipped], scatter [0–1]
8 Features	freq_norm, ndsm_mean, ndsm_std, building_density, tx_height, rx_h_mean, rx_h_std, urban_density
Materials	concrete, brick, glass, wet_ground, vegetation, water (6 materials)
Output	material_mlp_weights.npz + material_mlp_materials.json
In Sionna 2	Load .npz → compute 8 scene features → call predict_materials() → set εr/σ/S on RadioMaterials before simulation
NOTE	Fix applied (June 2026): dummy forward pass added to build Keras weights before training. Re-run Cell 16 to export valid weights.

Property	Detail
Portable?	■ Yes — designed for any city and frequency

2. Model Descriptions (continued)

2.4 Cell 15 — Physics-Informed Residual MLP (Thrane et al. 2020)

Type: Post-simulation RSSI correction. Takes Sionna RT path statistics as input and predicts a per-receiver dB correction. Does NOT modify scene materials. Based on Thrane et al. 2020 IEEE TVT "Model-Aided Deep Learning for Path Loss Prediction".

Property	Detail
Architecture	Input(45) → Dense(128,ReLU)+Dropout(0.2) → Dense(128,ReLU)+Dropout(0.2) + residual skip → Dense(64,ReLU) → Dense(1,linear)
45 Features	G1 Power (5): strongest path, total power, dominant ratio, power spread dB, path count norm G2 Delay (5): min/max/mean delay µs, RMS delay spread, coherence BW G3 Angular RX (5): mean/std azimuth+elevation, angular spread G4 Angular TX (5): same at TX side G5 Path types (5): LOS flag, specular/diffuse/diffraction counts, LOS power fraction G6 Vertex geometry (5): mean/max interaction height, mean/max path length G7 Reference PL (5): FSPL, 2D dist, log-distance, height diff, RX height AGL G8 Scene (5): building density, mean height, veg fraction, terrain std, urban flag G9 Morphology (5): reserved zeros
Training	56 train / 15 test (80/20 split). 500 epochs, Adam LR=1e-3, patience=80. Best epoch: 221. Early stopping + ReduceLROnPlateau.
RMSE result	RT only: 9.74 dB → RT + MLP: 4.83 dB (Δ = +4.92 dB, 50.5% improvement) N=15 test receivers
Output file	residual_mlp_915mhz.npz — contains: weights w0..wN, feat_mean, feat_std, frequency_hz
Portable?	■■ PARTIAL — MLP weights are fully portable (plain numpy arrays). Feature extraction code uses Sionna 0.19 paths 8-tuple API. Sionna 2 has a different paths structure → feature extraction must be adapted.

How to use in Sionna 2 DEM:

Step 1 — Run Sionna 2 DEM simulation → obtain RSSI_sim and paths per receiver.
Step 2 — Adapt _extract_features() to Sionna 2 paths API (paths.a, paths.tau, paths.phi_r, paths.theta_r, paths.vertices — accessed directly, not via 8-tuple index).
Step 3 — Load feat_mean, feat_std from residual_mlp_915mhz.npz. Normalise features: X_norm = (X - feat_mean) / feat_std.
Step 4 — Rebuild MLP architecture, load weights, call predict(X_norm). Apply: RSSI_final = RSSI_sim + delta.

2. Model Descriptions (continued)

2.5 Cell 14 — CNN + MLP Residual Corrector (nDSM height map)

Type: Post-simulation RSSI correction using spatial height information from the Digital Surface Model (nDSM). Fully portable across Sionna versions — uses only receiver positions and the nDSM raster file.

Property	Detail
Architecture	CNN branch: 64×64×1 nDSM patch → Conv(32,3×3,same) → MaxPool → Conv(64,3×3,same) → MaxPool → Conv(128,3×3,same) → GlobalAvgPool → Dense(64) MLP branch: 5 scalar feats → Dense(32) → Dense(32) Merge: Concat(96) → Dense(64) + Dropout(0.2) → Dense(1,linear) Total: 108,449 parameters
5 Scalar feats	dist_km/10, tx_height/100, rx_height/50, sin(azimuth), cos(azimuth)
nDSM patch	64×64 pixel = 64×64 m at 1 m/pixel. Clipped [0,120 m], normalised [0,1]. Centred on receiver UTM position. Source: ndsm.tif (EPSG:27700)
Training	60 train / 16 test. 150 epochs max, EarlyStopping patience=20. Best epoch: 23. Total training time: 6.9 s.
RMSE result	RT only: 14.82 dB → RT + CNN: 5.92 dB (Δ = +8.90 dB, 60% improvement) N=16 test receivers
Output file	cnn_residual_915mhz.h5 — full Keras model with weights (tf.keras.models.save_model)
Portable?	■ YES — no Sionna version dependency. Requires: receiver UTM positions + ndsm.tif

How to add directly to Sionna 2 DEM (copy-paste ready):

```
import tensorflow as tf, numpy as np, rasterio
from pyproj import Transformer

# Load saved CNN model
cnn = tf.keras.models.load_model("results/diff_rt/cnn_residual_915mhz.h5")

# Open nDSM (same file used in Cell 14)
ndsm_src = rasterio.open("ndsm.tif")
ndsm_arr = np.clip(ndsm_src.read(1).astype(np.float32), 0, 120) / 120.0
ndsm_tf  = Transformer.from_crs("EPSG:32630", "EPSG:27700", always_xy=True)

tx_pos = scene.transmitters[tx_name].position.numpy()

rssi_final = {}
for rx_name, rssi_sim in rssi_sim_dict.items():
    rx_pos  = scene.receivers[rx_name].position.numpy()
    ex, ey  = ndsm_tf.transform(rx_pos[0], rx_pos[1])  # UTM → EPSG:27700
    row, col = ndsm_src.index(ex, ey)
    r0, r1  = max(0,row-32), min(ndsm_arr.shape[0], row+32)
    c0, c1  = max(0,col-32), min(ndsm_arr.shape[1], col+32)
    patch   = np.zeros((64,64,1), dtype=np.float32)
    patch[(32-(row-r0)):(32+(r1-row)), (32-(col-c0)):(32+(c1-col)), 0] = \
            ndsm_arr[r0:r1, c0:c1]
    dx, dy  = rx_pos[0]-tx_pos[0], rx_pos[1]-tx_pos[1]
    scalar  = np.array([[np.sqrt(dx**2+dy**2)/1e4,
                         tx_pos[2]/100, rx_pos[2]/50,
                         np.sin(np.arctan2(dx,dy)),
                         np.cos(np.arctan2(dx,dy))]], dtype=np.float32)
    delta   = float(cnn.predict([patch[np.newaxis], scalar], verbose=0))
    rssi_final[rx_name] = rssi_sim + delta
```

3. Results Summary

Model	Notebook	RMSE before	RMSE after	Gain	Portable
Scalar Offset (Cell 10b)	sionna019	~22 dB	7.90 dB	+14 dB	■ No
Material Calib (Cell 11b)	sionna019	22.62 dB	21.94 dB	+0.67 dB	■■ Limited
MaterialMLP (Cell 16)	sionna019	—	—	Portable ver. of 11b	■ Yes
Residual MLP (Cell 15)	sionna019	9.74 dB	4.83 dB	+4.92 dB 50%	■■ Partial
CNN+MLP nDSM (Cell 14)	sionna019	14.82 dB	5.92 dB	+8.90 dB 60%	■ Yes

3.1 Literature Comparison

Method	RMSE	Source
Thrane et al. 2020 IEEE TVT (2.6 GHz, drive-test)	4–6 dB	Literature
NVLabs diff-RT material calib (Hoydis 2023, synthetic)	1–2 dB gain	Literature
This work — RT + Residual MLP (Cell 15, 915 MHz)	4.83 dB	■ Within range
This work — RT + CNN nDSM (Cell 14, 915 MHz)	5.92 dB	■ Within range
This work — material calib gain (Cell 11b, real-world)	+0.67 dB	■ Expected on real OSM

3.2 Key Finding

Cell 15 (4.83 dB, physics-feature MLP) outperforms Cell 14 (5.92 dB, CNN on nDSM) because RT-derived channel statistics (delay spread, angular spread, path count) are more informative predictors than height-map context alone — consistent with Thrane et al. 2020. However, Cell 14 is **fully portable to Sionna 2** with no API adaptation, making it the preferred model to add directly to sionna2_915mhz_dem_simulation.ipynb.

4. What Is Already Loaded in Sionna 2 DEM vs What Needs Adding

Source	File	Status in sionna2 Cell 4A	Action
Cell 10b	scalar_offset_915mhz.json	■ Loaded automatically	None — already works
Cell 11b	calibrated_materials_915mhz.json	■ Loaded automatically	None — already works
Cell 16	material_mlp_weights.npz	■■ Not yet wired	Add load block to Cell 4A
Cell 15	residual_mlp_915mhz.npz	■ Not in sionna2	Add post-process cell + adapt feature extraction to Sionna 2 API
Cell 14	cnn_residual_915mhz.h5	■ Not in sionna2	Add post-process cell — code in Section 2.5 is copy-paste ready

4.1 Recommended Next Steps

#	Task	Details	Est. time
1	Re-run Cell 16	Fix pushed (dummy forward pass). Re-run to export valid material_mlp_weights.npz.	5 min
2	Add Cell 14 CNN to sionna2 DEM	Copy code block from Section 2.5. No Sionna API changes needed.	15 min
3	Run sionna2 DEM full simulation	With Cell 11b materials + scalar offset already loaded. Get baseline RMSE.	20–30 min
4	Apply Cell 14 CNN correction	After simulation, apply delta from CNN → report final RMSE.	5 min
5	Adapt Cell 15 features for Sionna 2	Rewrite _extract_features() for Sionna 2 paths API (~50 lines). Then add to sionna2.	1 h